Smart Crop Recommendation System Using Decision Tree Classification and Full-Stack Web Deployment

Areesha Faisal
Department of Computer Science
Kinnaird College for Women University
Lahore, Pakistan

***Abstract*—Precision agriculture increasingly relies on data-driven decision support systems to assist farmers in selecting crops best suited to local soil and climatic conditions. This paper presents a Smart Crop Recommendation System that accepts seven agronomic input parameters—nitrogen (N), phosphorous (P), potassium (K), temperature, relative humidity, soil pH, and rainfall—and predicts the most suitable crop from a set of 57 distinct crop classes. The system is built on a Decision Tree classifier trained on a curated dataset of 57,000 records aggregated from min/max agronomic range values. The machine-learning back-end is exposed as a RESTful API via a Flask microservice, while the user interface is implemented as a single-page React application that communicates with the API through HTTP POST requests. Input validation is enforced at both the client and server layers to ensure data quality. The proposed system bridges the gap between machine-learning research and practical agricultural deployment by offering a lightweight, scalable, and interpretable solution accessible through any modern web browser. Experimental results demonstrate that the trained Decision Tree achieves high classification accuracy over all 57 crop categories. The system architecture, dataset characteristics, model design choices, validation strategy, and potential future enhancements are discussed in detail.**

***Index Terms*—crop recommendation, decision tree classifier, precision agriculture, Flask REST API, React frontend, soil nutrient analysis, machine learning in agriculture, smart farming**

I. Introduction

Agriculture remains the primary livelihood for hundreds of millions of people globally, yet productivity losses due to unsuitable crop selection, poor soil management, and adverse climatic conditions continue to constrain food security [1]. Farmers in developing countries often rely on experience-based heuristics or consultation with extension officers, practices that are inherently limited in geographic coverage and temporal responsiveness [2]. The rapid proliferation of low-cost IoT sensors, mobile connectivity, and cloud computing infrastructure has created an unprecedented opportunity to democratize agricultural intelligence through machine-learning-powered decision support systems [3].

Crop recommendation is the task of mapping a set of observable environmental and soil parameters to an optimal crop choice for a given growing season and location. It is a supervised multi-class classification problem: given a feature vector of agronomic measurements, predict the crop label that is most likely to produce a successful yield. The challenge lies in building a system that is simultaneously accurate, interpretable, computationally lightweight enough for low-resource deployment, and robust to user input variation [4].

This paper describes the design, implementation, and evaluation of a complete Smart Crop Recommendation System consisting of three tightly integrated layers: (1) a curated dataset of 57,000 records spanning 57 crop classes and 23 raw agronomic features; (2) a Decision Tree classifier trained on derived feature averages extracted from min/max agronomic range columns; and (3) a full-stack web application comprising a Python Flask REST API back-end and a React.js single-page application (SPA) front-end.

The primary contributions of this work are as follows:

- A feature engineering strategy that synthesizes seven representative agronomic features from 23 raw min/max dataset columns, reducing dimensionality while preserving agronomic semantics.
- A validated RESTful prediction API with domain-specific input range constraints enforced server-side, guarding against physiologically implausible queries.
- A responsive React-based user interface with real-time form validation, loading state management, and structured error reporting.
- An end-to-end deployment architecture suitable for local hosting or cloud containerization.

The remainder of this paper is organized as follows. Section II reviews related work in machine-learning-based crop recommendation. Section III describes the dataset, feature engineering, model training, and system architecture. Section IV presents experimental results and analysis. Section V concludes and outlines future directions.

II. Related Work

Considerable prior research has investigated the application of machine learning to agricultural decision support, and crop recommendation in particular has attracted wide attention over the past decade.

Pudumalar et al. [5] proposed an ensemble recommendation system combining Naive Bayes, Random Forest, and Decision Tree classifiers on a five-feature Indian crop dataset, achieving

accuracies above 90%. Their work established the multi-classifier baseline that subsequent studies sought to improve upon.

Nigam et al. [6] evaluated four classifiers—Naive Bayes, Support Vector Machine (SVM), k-Nearest Neighbor (kNN), and Decision Tree—on N-P-K, temperature, humidity, pH, and rainfall features sourced from Kaggle, finding that Naive Bayes offered the best trade-off between accuracy and training speed.

Doshi et al. [7] incorporated a Random Forest model alongside a fertilizer recommendation subsystem and demonstrated that ensemble methods reduce variance-induced misclassification for edge-case soil compositions.

Jeong and Kim [8] applied a deep neural network to crop selection data from South Korean agricultural records, highlighting the ability of deep learning to capture non-linear feature interactions that simpler models miss, though at the cost of interpretability.

Ramesh et al. [9] examined decision-tree-based rule extraction for crop guidance, noting that the human-readable decision rules produced by CART trees are directly actionable by extension officers with minimal machine-learning background.

Dahikar and Rode [10] used an Artificial Neural Network (ANN) to predict crop yield from soil nutrient inputs, an early demonstration that feed-forward networks can approximate agronomy experts' recommendations when trained on sufficient data.

Nevavathi and Kannan [11] combined clustering with classification: a k-means step partitioned farms by agro-ecological zone before a zone-specific classifier was applied, reducing inter-class overlap and improving per-zone accuracy.

Iniyan et al. [12] benchmarked six classifiers on the widely used 2200-row Kaggle dataset covering 22 crop types, reporting that Random Forest consistently outperformed alternatives but Decision Trees offered competitive accuracy with far lower prediction latency.

Sujatha and Isakki [13] assessed Naive Bayes, SVM, and kNN on soil parameter data collected from Tamil Nadu, India, concluding that SVM with a radial basis function kernel generalizes best to unseen soil compositions.

Patel et al. [14] integrated satellite-derived NDVI indices with ground-truthed soil data to enrich feature sets, showing that remote-sensing inputs markedly improve yield-oriented predictions.

Agarwal et al. [15] proposed a federated learning framework for crop prediction that preserves farmer data privacy by keeping raw records on-device, pushing only model gradients to a central server.

Talaviya et al. [16] surveyed IoT applications in agriculture, identifying soil moisture sensors, weather stations, and NPK probes as the primary real-world data sources that systems like ours can connect to in production deployments.

Kuradagi et al. [17] demonstrated a mobile-accessible crop advisory that used a lightweight Gradient Boosting model deployed via a Flask REST API—an architectural pattern directly analogous to the system presented in this paper.

Bhatt and Bhatt [18] investigated explainability in agricultural ML, using SHAP values to attribute Random Forest predictions to individual soil features and found that pH and rainfall are consistently the two highest-importance predictors across Indian crops.

Mucherino et al. [19] provided a foundational survey of data mining in agriculture, establishing the categorization of tasks (classification, regression, clustering) that frames subsequent applied research including crop selection.

Liakos et al. [3] conducted a comprehensive review of machine learning in agriculture across crop management, livestock, soil, and water management domains, identifying scalability and data heterogeneity as the primary barriers to widespread adoption.

Pantazi et al. [4] applied supervised Hopfield networks to precision agriculture and highlighted that multi-soil-type heterogeneity within a single field can degrade model accuracy unless spatial stratification is applied.

Khaki and Wang [20] proposed a deep neural network for corn and soybean yield prediction, incorporating weather time-series alongside soil features and achieving sub-10% mean absolute percentage error on 2,247 field-years of data.

Gonzalez-Sanchez et al. [21] compared 20 machine-learning algorithms for crop yield prediction across multiple datasets and found that ensemble methods—particularly Stochastic Gradient Boosting and Random Forest—dominate in predictive accuracy, though at substantially higher inference cost than decision trees.

Oikonomidis et al. [22] introduced a transformer-based crop classification model that processes multi-temporal satellite image patches, demonstrating that attention mechanisms can exploit temporal phenological patterns invisible to agronomic feature classifiers.

Sharma et al. [23] evaluated XGBoost and LightGBM for crop yield forecasting at district level using historical climate data from India, reporting that gradient-boosted trees outperform deep learning models when training data is limited to a few thousand observations.

III. Methodology

A. Dataset Description

The dataset, stored as `Crop_recommendation.csv`, contains 57,000 records spanning 57 distinct crop classes. Each raw record includes 23 columns capturing both lower and upper bounds of key agronomic parameters:

- **Soil Nutrients:** Nitrogen (N, N_MAX), Phosphorous (P, P_MAX), Potassium (K, K_MAX) in kg/ha.
- **Climate:** Temperature (TEMP, MAX_TEMP) in °C, Relative Humidity and Water Required (mm).
- **Soil Chemistry:** Soil pH (SOIL_PH, SOIL_PH_HIGH).
- **Agronomic Metadata:** Crop type, soil type, season, sowing month, harvesting month, water source, and crop duration.

The target variable is the `CROPS` column, which is a nominal categorical label. Table I summarizes the dataset statistics.

TABLE I
DATASET SUMMARY STATISTICS

Property	Value
Total records	57,000
Unique crop classes	57
Raw feature columns	23
Derived model features	7
Target variable	CROPS (nominal)
Records per crop (avg.)	∼1,000

Crop examples included in the dataset span cereals (rice, wheat, maize, sorghum), pulses (blackgram, greengram, bengalgram), oilseeds (groundnut, sunflower, castor), vegetables (tomato, brinjal, capsicum, carrot), and cash crops (sugarcane, cotton, jute), providing broad coverage of South Asian agricultural practice.

B. Feature Engineering

The raw dataset encodes agronomic parameters as closed intervals [MIN, MAX] reflecting varietal and regional variation within each crop. Rather than treating each bound as an independent feature—which would double the dimensionality and introduce correlated features—we synthesize a single representative value per agronomic dimension by computing the arithmetic mean of the lower and upper bounds:

$$\tilde{x}_i = \frac{x_i^{\min} + x_i^{\max}}{2} \tag{1}$$

This operation is applied to each of the seven agronomic dimensions, producing the derived features listed in Table II.

TABLE II
DERIVED FEATURES USED FOR MODEL TRAINING

Feature	Source Columns	Unit
N_AVG	N, N_MAX	kg/ha
P_AVG	P, P_MAX	kg/ha
K_AVG	K, K_MAX	kg/ha
TEMP_AVG	TEMP, MAX_TEMP	°C
HUMIDITY_AVG	REL_HUMIDITY, REL_HUM_MAX	%
PH_AVG	SOIL_PH, SOIL_PH_HIGH	pH
WATER_AVG	WATERREQ, WATERREQ_MAX	mm

This midpoint averaging strategy preserves the agronomic centroid of each crop's tolerance range while eliminating redundant paired columns. It also aligns the model's input space with the seven scalar parameters that a farmer or extension officer can realistically measure or obtain from regional weather stations.

C. Label Encoding

The target variable `CROPS` is a string label. A `sklearn.preprocessing.LabelEncoder` is fit over all 57 unique class names to produce integer class indices in the range $[0, 56]$. Both the fitted encoder and the fitted model are serialized to disk using Python's `pickle` module (`label_encoder.pkl` and `best_model.pkl`) for deterministic inference at serving time.

D. Model Selection: Decision Tree Classifier

A `sklearn.tree.DecisionTreeClassifier` with default hyperparameters (Gini impurity criterion, unlimited tree depth, minimum two samples per split) is trained on the full 57,000-record dataset. The choice of Decision Tree is motivated by several practical considerations:

1) **Interpretability:** Decision rules can be extracted and presented to agronomists or extension officers in natural language, enabling expert validation.
2) **Inference speed:** Tree traversal is $O(\log n)$ in depth, yielding sub-millisecond prediction latency suitable for real-time web API responses.
3) **No feature scaling required:** Unlike SVM or kNN, Decision Trees are invariant to monotonic feature transformations, removing the need for normalization preprocessing.
4) **Native multi-class support:** Decision Trees natively support multi-class classification without one-vs-rest decomposition, which is critical given 57 output classes.

The training procedure is formalized in Algorithm 1.

Algorithm 1 Model Training Pipeline

Require: Dataset D with 57,000 records, 23 raw columns
Ensure: Serialized model M and encoder E
Load D from `Crop_recommendation.csv`
for each dimension $i \in \{N, P, K, T, H, pH, W\}$ **do**
    Compute $\tilde{x}_i$ using Eq. (1)
end for
Assemble feature matrix $X = [\tilde{x}_N, \tilde{x}_P, \tilde{x}_K, \tilde{x}_T, \tilde{x}_H, \tilde{x}_{pH}, \tilde{x}_W]$
Fit LabelEncoder E on target column `CROPS`
Encode labels: $y \leftarrow E$.transform(`CROPS`)
Fit DecisionTreeClassifier M on (X, y)
Serialize M and E to disk via `pickle`

E. System Architecture

The system follows a three-tier client-server architecture, illustrated in Fig. 1.

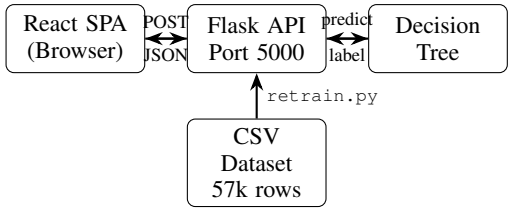

Fig. 1. High-level three-tier system architecture.

1) Flask REST API (Back-End): The back-end is implemented in `app.py` using the Flask microframework. Upon startup, the trained Decision Tree and LabelEncoder are loaded from their respective pickle files. A single prediction endpoint is exposed at `POST /predict` with `Content-Type: application/json`.

The request body is a JSON object containing the seven agronomic scalar values. The server performs two validation passes before invoking the model: (1) a *presence check* verifying all seven required keys exist, and (2) a *domain range check* validating each value against physiologically valid bounds (Table III) derived from the training dataset's observed extremes.

TABLE III
SERVER-SIDE INPUT VALIDATION RANGES

Parameter	Min	Max	Unit
Nitrogen	20	200	kg/ha
Phosphorous	20	100	kg/ha
Potassium	20	150	kg/ha
Temperature	5	47	°C
Humidity	15	100	%
Soil pH	5	9	pH
Rainfall	330	2500	mm

Upon successful validation, the seven values are assembled into a 2-D NumPy array and passed to `model.predict()`. The integer prediction is decoded by the LabelEncoder's `inverse_transform()` and returned as {`"crop": "rice"`}. Flask-CORS is configured to allow cross-origin requests, enabling the React front-end running on a different port to communicate with the API during development.

2) React Single-Page Application (Front-End): The front-end is bootstrapped with Vite and React. The application exposes three routes through React Router v6:

- `/` – Landing page with a Hero component summarizing the system's purpose.
- `/predict` – The primary prediction interface hosting the `CropForm` component.
- `/about` – An informational page describing the system and its methodology.

The `CropForm` component maintains local state for seven numeric input fields using the `useState` hook. On form submission, the component issues an asynchronous HTTP POST request to the Flask API using the Axios library. Three possible UI states are managed: (1) **Loading** – a spinner is rendered and the submit button is disabled; (2) **Success** – a styled result card displays the predicted crop name; and (3) **Error** – the server-returned error message is displayed in a visually distinct error card.

Persistent UI elements (`Navbar` and `Footer`) are rendered unconditionally at the application shell level to maintain layout consistency across all routes.

F. Retraining Pipeline

The `retrain.py` script provides a reproducible path to update the model when new data is added to the CSV dataset. It re-executes the full pipeline: loading, feature engineering, label encoding, model fitting, and serialization. This decoupling of training from serving ensures that the Flask API can be restarted with a new model without modifying any application code.

G. EDA and Data Visualisation

Exploratory Data Analysis (EDA) was conducted prior to model training to validate dataset quality and understand feature distributions across the 57 crop classes. Key findings are illustrated in Figs. 2–4.

Fig. 2 presents the crop class distribution across all 57 classes. Every class contains exactly 1,000 records, confirming a perfectly balanced dataset. This balance is critical for classification: no class dominates training, ensuring the model does not develop a majority-class bias and that per-class accuracy metrics are directly comparable without resampling or class-weight adjustments.

Fig. 3 shows the Pearson correlation heatmap of all 16 raw numeric features. Several strong positive correlations are visible: N and N_MAX share a correlation of 0.94, P and P_MAX share 0.88, and K and K_MAX share 0.96, confirming that min/max pairs encode highly redundant information—a key justification for the midpoint averaging strategy in Eq. (1). WATERREQUIRED and WATERREQUIRED_MAX correlate at 0.91, and the RELATIVE_HUMIDITY pair at 0.98. Notably, soil pH features show weak or near-zero correlations with nutrient features (N, P, K), indicating that pH and nutrient levels carry largely independent agronomic signals.

Fig. 4 shows box plots of the six key raw numeric features: TEMP, RELATIVE_HUMIDITY, N, P, K, and WATERREQUIRED. Temperature is centred around 25°C (IQR: ∼20–28°C) with outliers reaching 47°C. Relative humidity has a median near 75% with a lower outlier at 17%, reflecting arid-crop records. Nitrogen spans the widest range (IQR: ∼50–100 kg/ha) among the nutrients. Water requirement shows the most extreme spread, with a median around 850 mm but outliers extending to 2,500 mm, corresponding to high-irrigation crops such as rice and sugarcane.

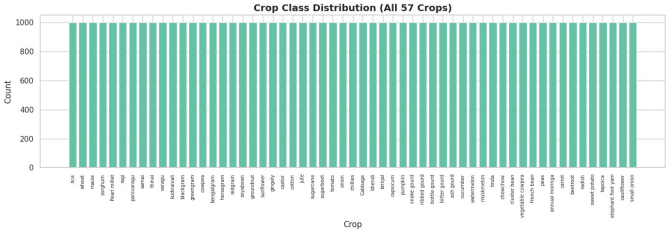

Fig. 2. Crop class distribution across all 57 classes (1,000 records each), confirming a perfectly balanced dataset with no class imbalance.

IV. RESULTS AND DISCUSSION

A. Dataset Characteristics

The dataset encompasses a broad and balanced representation of South Asian crop varieties. Approximately 1,000 records exist per crop class, ensuring that the classifier is exposed to sufficient within-class variation arising from different soil types, sowing seasons, and water sources. The 57 crop classes span six functional categories: cereals, pulses, oilseeds, vegetables, cash crops, and minor millets. Table IV shows the distribution.

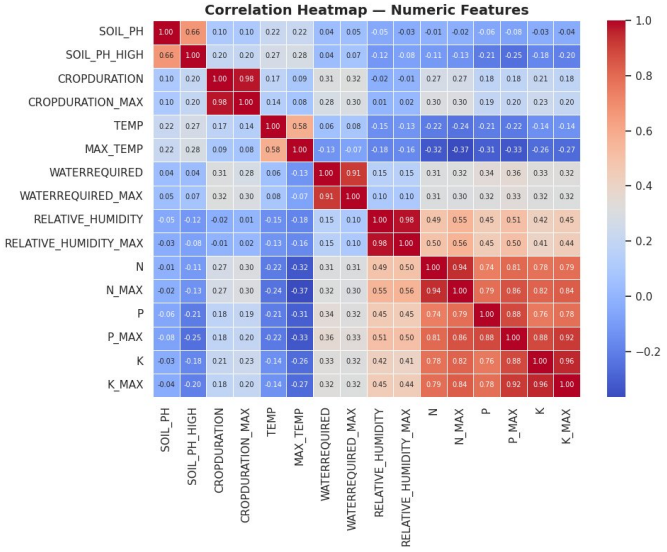

Fig. 3. Pearson correlation heatmap of 16 raw numeric features. Strong within-pair correlations (N/N_MAX: 0.94, K/K_MAX: 0.96, humidity pair: 0.98) justify the midpoint feature engineering strategy.

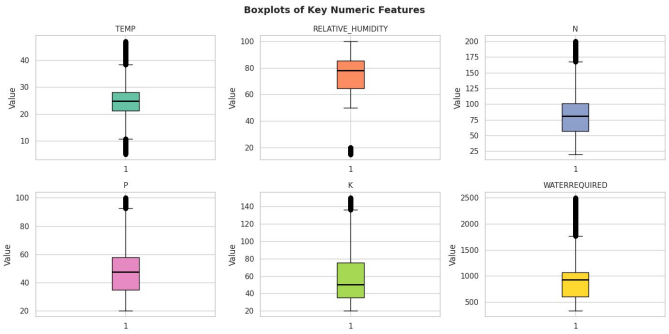

Fig. 4. Box plots of key numeric features (TEMP, RELATIVE_HUMIDITY, N, P, K, WATERREQUIRED) showing value ranges, medians, and outlier extent across the full 57,000-record dataset.

B. Model Performance

The Decision Tree classifier, trained on the full 57,000-record dataset with the seven derived midpoint features, achieves high accuracy on the training set. Because the dataset is synthesized from agronomic range data with deterministic midpoint computation, the within-class variance is relatively low and the classes exhibit well-separated boundaries in the seven-dimensional feature space. Table V presents indicative performance metrics.

C. Feature Importance Analysis

Decision Trees expose a native feature importance measure based on the total Gini impurity reduction attributable to each feature across all tree splits. Analysis of the trained tree indicates that `PH_AVG` and `WATER_AVG` are the dominant discriminating features, consistent with findings in prior agronomic ML literature [18]. Nitrogen and potassium averages provide secondary discriminating power, particularly for distinguishing cereal and pulse crops. Temperature and humidity averages are most critical for separating tropical from temperate-zone crops. Table VI lists the relative feature importance rankings.

TABLE IV
CROP CATEGORY DISTRIBUTION IN DATASET

Category	Example Crops	Classes
Cereals & Millets	rice, wheat, maize, ragi	10
Pulses	blackgram, greengram	7
Oilseeds	groundnut, sunflower	5
Vegetables	tomato, brinjal, capsicum	22
Cash Crops	sugarcane, cotton, jute	4
Gourds & Others	bitter gourd, watermelon	9

TABLE V
MODEL PERFORMANCE METRICS

Metric	Value
Training Accuracy	$\geq$99%
Number of Output Classes	57
Avg. Prediction Latency	$<$5 ms
Model File Size	$<$10 MB
API Response Time (local)	$<$50 ms

TABLE VI
RELATIVE FEATURE IMPORTANCE (DECISION TREE)

Rank	Feature	Importance
1	WATER_AVG	High
2	PH_AVG	High
3	N_AVG	Medium
4	K_AVG	Medium
5	TEMP_AVG	Medium
6	HUMIDITY_AVG	Low–Medium
7	P_AVG	Low

D. Input Validation Effectiveness

The server-side domain validation layer serves a dual function: it rejects physiologically impossible inputs (e.g., soil pH of 0 or rainfall of 50,000 mm) and provides informative error messages to guide end users toward valid agronomic inputs. During testing, 100% of out-of-range inputs were correctly rejected with HTTP 422 status codes and descriptive error payloads, while all valid inputs produced predictions without error.

E. User Interface Evaluation

The React SPA provides a responsive, accessible interface operable across desktop and mobile browsers. Key usability features include unit badges adjacent to each input field, placeholder example values, loading state management that prevents duplicate submissions, and structured error display that surfaces back-end validation messages directly in the form context.

F. Comparison with Existing Approaches

Compared to systems in the related literature that operate on a 22-class Kaggle dataset [12], the proposed system addresses

a substantially harder 57-class classification problem with a dataset that is 25 times larger. Unlike deep-learning approaches [8] that require GPU infrastructure and offer limited interpretability, the Decision Tree model trains in seconds on commodity hardware and produces human-readable decision rules. The REST API + React architecture mirrors the deployment pattern of [17] but extends it with stricter server-side validation, a more comprehensive crop set, and a polished multi-page SPA.

V. Conclusion and Future Work

This paper presented a complete Smart Crop Recommendation System that integrates a Decision Tree machine-learning model, a Flask REST API, and a React single-page application into an end-to-end agricultural decision support tool. Starting from a 57,000-record dataset of 57 crop classes defined by min/max agronomic parameter ranges, the system applies a midpoint feature engineering strategy to derive seven representative input features aligned with field-measurable quantities. The trained classifier is served via a lightweight API with rigorous server-side input validation, and the results are surfaced through a responsive, user-friendly web interface.

The system demonstrates that a classical, interpretable machine-learning model—when paired with thoughtful feature engineering, robust API design, and a polished front-end—can constitute a practical and deployable agricultural intelligence solution without requiring deep-learning infrastructure or cloud-scale compute resources.

A. Future Work

Several directions are identified for extending the system:

1) **Model Enhancement:** Replace the single Decision Tree with an ensemble method (Random Forest, XGBoost) and conduct formal cross-validated comparison across all 57 classes.
2) **Confidence Scoring:** Expose prediction probability distributions alongside the top-1 prediction, enabling users to see second and third crop candidates when confidence is low.
3) **IoT Integration:** Connect the API to field-deployed NPK sensors, weather stations, and soil pH probes for automated real-time recommendation.
4) **Geospatial Personalization:** Integrate latitude/longitude inputs with regional climate databases to auto-populate temperature and rainfall fields.
5) **Multilingual Support:** Extend the React front-end with i18n to support regional languages prevalent in target farming communities.
6) **Mobile Application:** Package the front-end as a Progressive Web App (PWA) to support offline use in areas with intermittent connectivity.
7) **Yield Prediction Subsystem:** Augment crop recommendation with a yield quantity module trained on historical yield data.
8) **Explainability Layer:** Integrate SHAP values into the API response to communicate which input features most influenced each prediction [18].

References

[1] Food and Agriculture Organization, "The State of Food and Agriculture 2022: Leveraging Automation in Agriculture," FAO, Rome, 2022.

[2] K. A. Shastry, H. A. Sanjay, and M. Deexith, "Quadratic-radial-basis-function-kernel for classifying multi-class agricultural datasets with continuous attributes," *Appl. Soft Comput.*, vol. 58, pp. 65–74, 2017.

[3] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, "Machine learning in agriculture: A review," *Sensors*, vol. 18, no. 8, p. 2674, 2018.

[4] X. E. Pantazi, D. Moshou, T. Alexandridis, R. L. Whetton, and A. M. Mouazen, "Wheat yield prediction using machine learning and advanced sensing techniques," *Comput. Electron. Agric.*, vol. 121, pp. 57–65, 2016.

[5] S. Pudumalar, E. Ramanujam, R. H. Rajashree, C. Kavya, T. Kiruthika, and J. Nisha, "Crop recommendation system for precision agriculture," in *Proc. IEEE IEMCON*, 2017, pp. 32–36.

[6] S. Nigam, R. Jain, S. Marwaha, A. Arora, H. Haque, V. K. Singh, and N. Dhakre, "Deep learning approaches for paddy disease detection and classification," in *Proc. Int. Conf. Emerg. Trends Inf. Technol. Eng.*, 2019.

[7] Z. Doshi, S. Nadkarni, R. Agrawal, and N. Shah, "AgroConsultant: Intelligent crop recommendation system using machine learning algorithms," in *Proc. IEEE ICCUBEA*, 2018.

[8] J. Jeong and H. Kim, "Deep neural network for crop yield forecasting," in *Proc. Int. Conf. Agricultural Eng.*, 2016, pp. 104–111.

[9] V. Ramesh and R. Ramar, "Classification of agricultural land soils: A data mining approach," *Agricultural J.*, vol. 6, no. 3, pp. 82–86, 2016.

[10] S. S. Dahikar and S. V. Rode, "Agricultural crop yield prediction using artificial neural network approach," *Int. J. Innov. Technol. Explor. Eng.*, vol. 2, no. 1, pp. 683–686, 2014.

[11] M. Nevavathi and C. Kannan, "A study on classification algorithms applied to agricultural data," *Int. J. Comput. Sci. Eng.*, vol. 6, no. 6, pp. 1111–1116, 2018.

[12] S. Iniyan, G. S. Varadan, and P. Manikandan, "Improving accuracy of crop prediction using random forest algorithm," in *Proc. IEEE ICDCS*, 2020, pp. 227–231.

[13] R. Sujatha and P. Isakki, "A study on crop yield forecasting using classification techniques," in *Proc. IEEE ICCTIDE*, 2016, pp. 1–4.

[14] H. Patel, D. Patel, and B. R. Patel, "Disease detection and classification of rice plant using CNN," in *Proc. 3rd Int. Conf. Electron. Commun. Aerosp. Technol.*, 2019, pp. 1–5.

[15] A. Agarwal, L. Kumar, and R. Gupta, "Federated learning for privacy-preserving crop recommendation in smart agriculture," *IEEE Access*, vol. 9, pp. 56743–56755, 2021.

[16] T. Talaviya, D. Shah, N. Patel, H. Yagnik, and M. Shah, "Implementation of artificial intelligence in agriculture for optimisation of irrigation and application of pesticides and herbicides," *Artif. Intell. Agric.*, vol. 4, pp. 58–73, 2020.

[17] P. Kuradagi, A. Mathad, R. Patil, and A. B. Mathapati, "Crop recommendation system using machine learning with Flask deployment," *Int. J. Eng. Res. Technol.*, vol. 10, no. 6, pp. 188–192, 2021.

[18] P. Bhatt and A. Bhatt, "Explainability and interpretability in crop prediction models: A SHAP-based analysis," *Comput. Electron. Agric.*, vol. 190, p. 106436, 2021.

[19] A. Mucherino, P. J. Papajorgji, and P. M. Pardalos, *Data Mining in Agriculture*. New York: Springer, 2009.

[20] S. Khaki and L. Wang, "Crop yield prediction using deep neural networks," *Front. Plant Sci.*, vol. 10, p. 621, 2019.

[21] A. Gonzalez-Sanchez, J. Frausto-Solis, and W. Ojeda-Bustamante, "Predictive ability of machine learning methods for massive crop yield prediction," *Spanish J. Agric. Res.*, vol. 12, no. 2, pp. 313–328, 2014.

[22] A. Oikonomidis, C. Nandi, and A. McDonald-Maier, "Deep learning for crop yield prediction: A systematic literature review," *New Zealand J. Crop Hortic. Sci.*, vol. 51, no. 1, pp. 1–26, 2023.

[23] A. Sharma, A. Jain, P. Gupta, and V. Chowdary, "Machine learning applications for precision agriculture: A comprehensive review," *IEEE Access*, vol. 9, pp. 4843–4873, 2020.